

《算法分析与设计》第 4 次作业 *

姓名：谢鹏宇 学号：71122135 成绩：_____

算法分析题

题目 1：假设有 n 个活动 $a_1, a_2, \dots, a_n$ 须要使用焦廷标馆。第 i 个活动 ($1 \leq i \leq n$) 须要连续使用 t_i 时间。这些活动没有定死的开始或终止时刻，但都希望从 $t = 0$ 开始。如果在时刻 $t > 0$ 开始，那么必须要付出额外的开销。这个开销与开始时刻 t 成正比。为简单起见，就把开始时刻 t 作为开销。请设计一贪心算法来找到一个最佳调度使总的开销最小。也就是说，如果活动 a_i 开始时刻是 s_i ，算法要找到一个两两兼容的调度使 $\sum_{i=1}^n s_i$ 最少，即找到活动的最佳开始时间。请证明算法正确性和分析复杂度。

答：

将活动的使用时间 t_i 排序，先安排活动使用时间短的，再安排活动使用时间长的，活动与活动之间无间隔时间。

```
void minCost(){
    quickSort(vector<int>& arr, int l1, int n);
}
```

时间复杂度为排序所需的时间复杂度，为 $O(n \log n)$

证明：

设该活动的最优解为 $A = \{a_1, a_2, \dots, a_n\}$ ，且满足每个活动所需时间依次增加

令活动集为 $S = \{s_1, s_2, \dots, s_n\}$

归纳基础：当 $k=1$ 时，若最优解不包含 a_1 ，则存在最优解 $A^* = \{a_j, \dots, a_n\}$ 且 $j \neq 1$ ，因为 a_1 为所有活动中需要时间最短的活动，所以最优解 A^* 中 a_{j+1} 开销必然大于 a_1 作为第一个活动的最优解 A

归纳步骤：当算法执行到第 k 步时，最优解包含了 $a_1, a_2, \dots, a_k$ ，剩下的活动为其余所有的活动，即 $A = \{a_1, a_2, \dots, a_k\} \cup B$ ，而 B 一定为剩下活动中的最优解，否则若 B^* 开销小于 B ，则原解 A 非最优解，与 A 为最优解矛盾。

*要求：1、分析题请用书面化语言给出详细分析过程。

题目 2: 假设一长途汽车线路上有 n 个站点并从 1 开始顺序编号为 $1, 2, \dots, n$, 其中起点站为 1, 终点站为 n 。旅客可以从任一个站 i 上车, $1 \leq i \leq n-1$, 到下面任一站 j 下车, $i < j \leq n$ 。假设已知 $p(i, j)$ 为需要在站 i 上车, 在站 j 下车的旅客人数 ($1 \leq i < j \leq n$)。又假设每辆公共汽车在每个站都停靠, 并可载客 30 人。请设计一个 $O(n^2)$ 算法来确定至少需要多少辆汽车可以满足所有旅客需要。

答:

```
void minNumber(){
    int counter = 1;
    int capacity = 0;
    int number = 0;
    for(int i=1; i<n; ++i){
        for(int j=i+1; j<=n; ++j)
            capacity+=p[i][j];
        for(int j=1; j<i; ++j)
            capacity-=p[j][i];
        if(number<capacity)
            number = capacity;
    }

    counter = capacity/30 + capacity%30;
}
```

最终至少需要的车辆数为 counter

题目 3: 假设我们开一部卡车从提瓦特大陆 A 到海拉鲁大陆 B , 沿路经过一些采原石场。为方便起见, 假定一共有 n 个采原石场, 顺序编号为 1 到 n , 其中采原石场 A 为第 1 号, 采原石场 B 为第 n 号。在每个采原石场 i , $1 \leq i \leq n$, 我们可以买原石, 价格已知为 $B[i]$ (元/吨), 也可以卖原石, 卖价为 $S[i]$ 。我们希望在某个采原石场 i 买原石, 然后在某采原石场 j 卖掉, $j \geq i$, 使得赚的钱最多, 即 $M = S[j] - B[i]$ 最大。请设计一个 $O(n)$ 的贪心算法找出这两个采原石场 i 和 j 并报告这个最大差价 $M = S[j] - B[i]$ 。我们规定, 车子只能向前开, 不可以往回开。你可以在同一采原石场买和卖, 但只能买一次和卖一次。如果这个最大差价是负数也给予报告, 说明最少会亏多少。

答: 将所有采原石场遍历一遍, 记录买价最低的场和卖价最高且在买的场或之后, 两个差值为最大差价。

```
void maxMoney(){
    int min=B[1], max=S[1];
```

$B[] S[]$

2, 2 1, 2 2, 1 3, 1

```
for(int i=2;i<=n;++i){
    if(min>B[i]){
        min=B[i];
        max=S[i];
    }
    if(max<S[i]){
        max=S[i];
    }
}
int money=max-min;
}
```

1 - 2
= -1

最终最大差价为 money